Secure File System Deliverable

Aaron Brady, Isaac Vissers, Jason Gillanders

Abstract

This document presents our design of a Secure File System (SFS) that will demonstrate our knowledge of the lecture content from ECE 422. The motivation behind the system is to allow internal users to store data on an untrusted file server. The system will support an admin user, group management, authenticated access, encrypted file and directory storage and permission controls. The system will be implemented as a monolithic Python application that supports an interactive command-line interface and provides secure and reliable file management within a single machine environment.

Introduction

The objective of this project is to design and implement a Secure File System (SFS) that enables internal users to securely store data on an untrusted file server. Although external users may be able to observe that files and directories exist, they must not be able to read file names, access file contents, or modify stored data without detection. The system therefore enforces authentication, encrypted storage, controlled file sharing, and integrity verification to protect user data.

The design of the SFS is guided by the principles of the CIA triad. Confidentiality is achieved by encrypting all file and directory names and contents on disk, ensuring that only authenticated and authorized users can access protected resources. Integrity is enforced through cryptographic mechanisms that detect tampering or corruption of files and metadata, with users notified of any integrity violations upon login. Availability is maintained by providing authenticated users reliable access to their files and directories through a structured permission model and an interactive command-line interface.

The system supports multiple users and groups, and controlled file sharing through permissions. Implemented as a monolithic Python application with an interactive CLI, the Secure File System demonstrates how core security principles can be applied to file system design while operating within an untrusted single-machine environment.

Planned Designs

Technologies

We plan on implementing this program in Python because it simplifies the development process. We all have experience with Python, and it has well maintained libraries for implementing CLI and encryption.

- Library for interactive shell
    - Cmd

        - Built in library
        - Well documented
        - This is the exact use case for this library
    - Cryptography (https://pypi.org/project/cryptography/)
        - Well documented
        - Implements RSA key encryption, which is what we learned in class

Methodologies

We plan on using Agile and Test Driven Development methodologies for this project.

We believe that Agile is well suited for this project as the implementation can be broken down into chunks which can be implemented in an iterative manner. (e.g., authentication, encryption, cli, user operations, file access options, tamper detection). Implementing these in iterations allows us to have working code early, validate our code early, and address issues early. We especially think that this is a good methodology because we have never implemented encryption before and we want the ability to change our requirements, design and implementation if that is necessary.

We believe that test driven development will be effective for this project because correctness, security, and reliability of the system are crucial and this allows us to cover edge cases, ensure the validity of the encryption algorithm we implement, and that the system fails safety.

Tools

We plan on using GitHub for version control with GitHub actions for our CI/CD pipeline. In our CI pipeline we want to use pytest to run our unit tests, ruff for code linting and formatting. For our CD portion of the pipeline (release on tag) we want to generate the artifacts required for a user to run our project, we would like to combine the python code and libraries into a single executable that can be ran without directly installing independencies, as well as a readme explaining how to use the program which will be released via github releases.

We chose this toolchain because it is lightweight, well-supported, and integrates seamlessly with our development workflow for the purposes of a demo. GitHub Actions provides automated validation of code quality and correctness, which is especially important for a security-focused system where regressions could introduce vulnerabilities. The release-on-tag strategy ensures that only tested and stable versions are distributed, promoting reproducibility and reliability for users and TAs who will be present for our demo.

User Stories

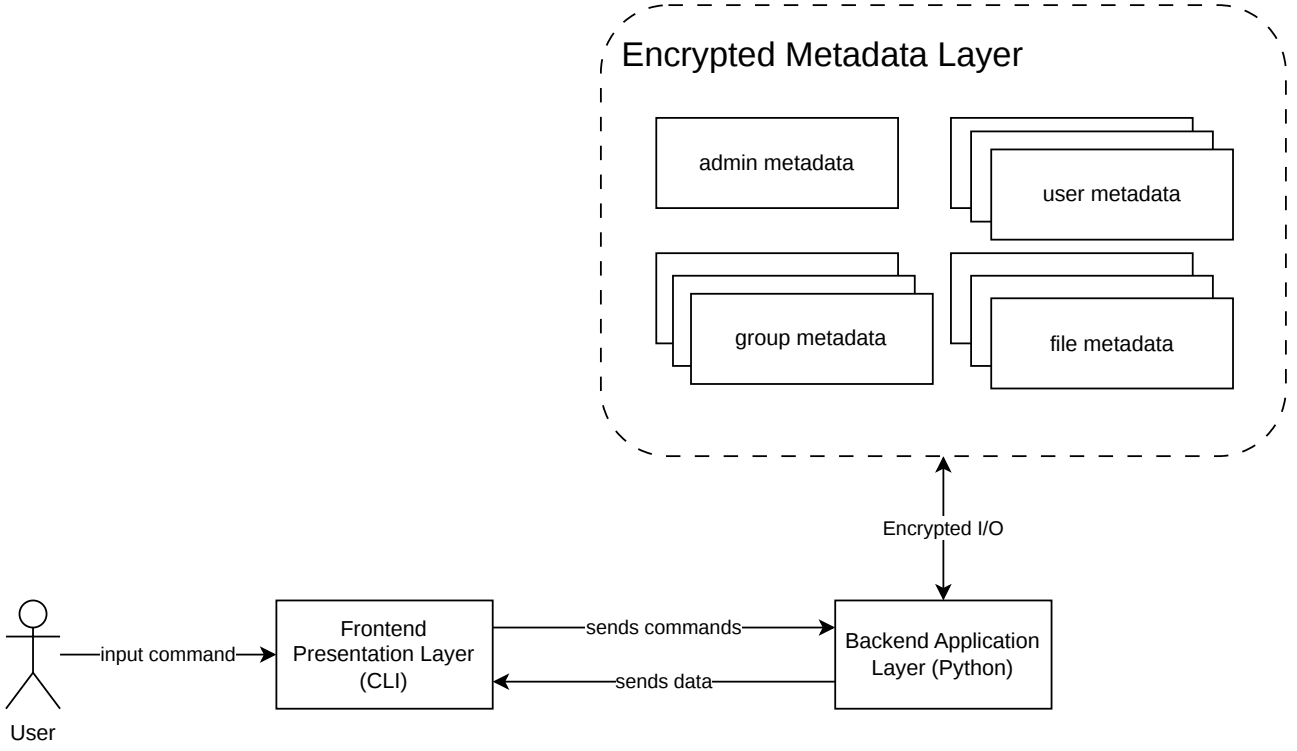
- **US-01:** As an administrator, I want to create user accounts so that authorized individuals can access the Secure File System.
  Acceptance Criteria: An authenticated administrator can create a new user account with a

unique username and password, after which the user has a provisioned home directory and can successfully log in, with the password stored securely and not in plaintext.

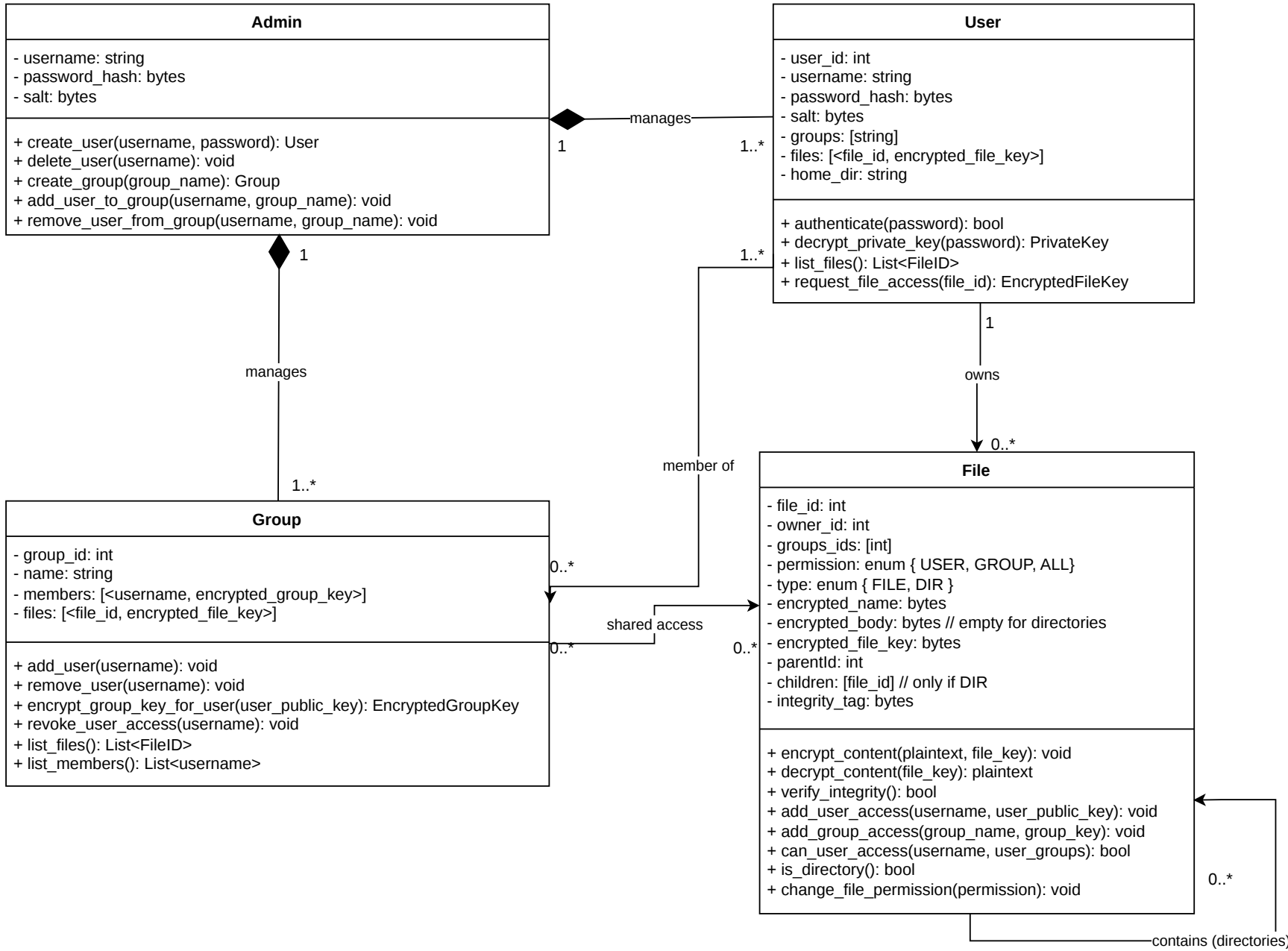
- **US-02:** As an administrator, I want to create groups so that users can be organized under shared access boundaries.
  Acceptance Criteria: An authenticated administrator can create a new group that is stored in the system and available for assigning users.
- **US-03:** As an administrator, I want to add users to groups so that I can manage group-based access control.
  Acceptance Criteria: An authenticated administrator can add an existing user to an existing group, after which the user inherits that group's access permissions.
- **US-04:** As an administrator, I want to remove users from groups so that I can revoke their group-based access.
  Acceptance Criteria: An authenticated administrator can remove a user from a group, after which the user no longer has access to group-protected resources.
- **US-05:** As an administrator, I want to view the usernames of all registered users so that I can manage the system, but I should not be able to view user passwords.
  Acceptance Criteria: An authenticated administrator can view a list of all registered usernames, but cannot view or retrieve any user passwords.
- **US-06:** As a user, I want to securely authenticate to the system so that only authorized users can access files.
  Acceptance Criteria: A user can log in using valid credentials, and access to the system is denied if authentication fails.
- **US-07:** As a user, I want an automatically provisioned home directory so that I have a private workspace.
  Acceptance Criteria: Upon successful account creation, the system automatically creates a home directory accessible only according to the defined permission model.
- **US-08:** As a user, I want to create files so that I can store data on the system.
  Acceptance Criteria: An authenticated user can create a file within an authorized directory, and the file is stored encrypted on disk.
- **US-09:** As a user, I want to read files I have permission to access so that I can retrieve stored data.
  Acceptance Criteria: An authenticated user can read the contents of a file only if permitted by its access controls, and the correct decrypted content is returned.
- **US-10:** As a user, I want to write to files I have permission to modify so that I can update stored data.
  Acceptance Criteria: An authenticated user can modify a file only if permitted by its access controls, and the updated content is stored encrypted on disk.
- **US-11:** As a user, I want to rename files so that I can organize my workspace.
  Acceptance Criteria: An authenticated user can rename a file they have permission to modify, and the updated filename remains encrypted on disk.
- **US-12:** As a user, I want to delete files I own or have permission to remove so that I can manage my storage.
  Acceptance Criteria: An authenticated user can delete a file only if permitted by its access controls, and the encrypted file data is removed from disk.

- **US-13:** As a user, I want to create and navigate directories within my home directory so that I can manage files hierarchically.
  Acceptance Criteria: An authenticated user can create and navigate directories within permitted locations, and directory names are stored encrypted on disk.
- **US-14:** As a file owner, I want to set permissions (user/group/all) on files and directories so that I can control who can read, write, or access them.
  Acceptance Criteria: A file owner can set user/group/all permissions on a file or directory, and those permissions are enforced by the system.
- **US-15:** As a user, I want group-based access controls enforced so that I can see files of users in my group (with encrypted names unless I have access), and not see files of users outside my group.
  Acceptance Criteria: A user can view the existence of files belonging to users in the same group according to permission rules, while files of users outside their groups are not visible.
- **US-16:** As a user, I want all files and directories stored encrypted on disk so that no one can access them without authenticating.
  Acceptance Criteria: All file and directory data stored on disk is encrypted such that it cannot be read without successful authentication through the SFS.
- **US-17:** As a user, I want file and directory names and contents encrypted at rest so that external users cannot read them directly from disk.
  Acceptance Criteria: File and directory names and contents appear as encrypted data when viewed directly on disk outside the SFS application.
- **US-18:** As a user, I want the system to detect tampering or corruption of file or directory names and contents so that integrity violations are identified.
  Acceptance Criteria: If any file or directory data is modified outside the SFS, the system detects the integrity violation during verification.
- **US-19:** As a user, I want to be notified of any integrity failures upon login so that I immediately know if my data was modified.
  Acceptance Criteria: Upon login, the system checks for integrity violations and notifies the user if any corruption or tampering is detected.

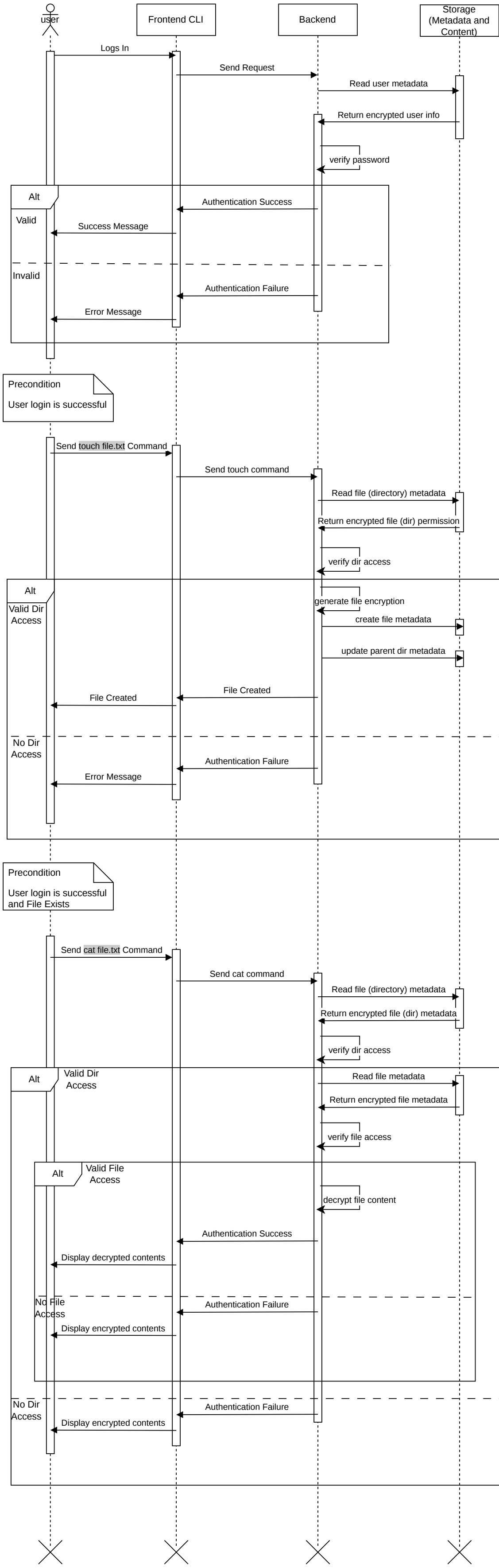
Architecture Diagram



Class Diagram



Sequence Diagram - User



Sequence Diagram - Admin